GIAN ROHR
23.3.2026
3SA

Inhaltsverzeichnis

1. Erklärung der Themawahl 1
2. Planung und Vorbereitung 3
3. Einzelne Arbeitsschritte 4
3.1 Grundlagen Java lernen 4
3.2 Einarbeiten in JavaFX 5
3.3 Applikation implementieren & testen 6
4. Rückblick / Fazit 8
5. Quellenverzeichnis 9

1. Erklärung der Themawahl

Am Anfang wollte ich etwas Handwerkliches machen. Ich fand die Vorstellung, etwas Eigenes aus Holz oder Metall herzustellen, sehr spannend. Doch je länger ich darüber nachdachte, desto mehr merkte ich, dass ich nichts fand, dass mir wirklich Spass machen würde oder ich machen kann, da ich handwerklich nicht wirklich begabt bin. Obwohl es Ideen gab, welche ich interessant fand, passten sie entweder nicht richtig zu mir oder waren zu kompliziert zum Umsetzen.

Danach kam ich auf die Idee, ein Video zu filmen. Am Anfang fand ich den Gedanken spannend, doch mir fehlten die Ideen sowie die Motivation. Da ich keine guten Ideen für das Video gefunden habe, gab ich diese Idee ebenfalls auf.

Da ich mich schon immer für Technik interessiert habe, war meine nächste Idee, etwas in diesem Bereich zu machen. Es war ebenfalls eine gute Idee, da es etwas leicht Handwerkliches, aber auch Technisches wäre. Zuerst hatte ich die Idee etwas zu bauen, zum Beispiel ein ferngesteuertes Auto oder eine Lampe, die sich automatisch einschaltet, sobald es dunkel wird oder man den Raum betritt. Obwohl ich diese Ideen zuerst für gut hielt, merkte ich schnell, dass sie ziemlich aufwendig wären und ich niemanden kenne, der Ahnung davon hat. Allein die Beschaffung der Materialien hätte mehrere Stunden gedauert. Wahrscheinlich hätte ich auch einiges bestellen müssen. Ausserdem hätte es das Risiko gegeben, dass etwas nicht funktioniert oder nicht kompatibel ist und ich nochmals von vorne beginnen hätte müssen. Das alles fand ich zu kompliziert und aufwendig.

Schliesslich brachte mein Vater mich auf eine sehr gute Idee, ein Spiel zu programmieren. Dieser Gedanke gefiel mir, da ich nicht mehr viel Zeit hatte mich für ein Projekt zu entscheiden und ich schon immer mal ein Spiel programmieren wollte. Als erstes überlegte ich mir, was für ein Spiel ich programmieren möchte. Ich musste entscheiden, ob ich ein komplett eigenes Spiel erstellen möchte oder ob ich meine eigene Version eines bereits bestehenden Spiels erstellen sollte. Ein eigenes Spiel zu erfinden, klang zwar spannend, aber ich hatte keine Ideen, was für ein Spiel es werden sollte.

Mein Vater brachte mich schliesslich auf eine gute Idee. Er schlug vor, statt eines modernen und sehr aufwendigen Spiels, eher ein älteres und einfacheres Spiel zu programmieren. Dieser Vorschlag überzeugte mich.

Das einzige Problem war nun, dass ich noch nicht programmieren konnte. Ich habe bisher noch nie etwas in diesem Bereich gemacht und wusste nicht genau, wo ich anfangen sollte. Zum Glück hatte mein Vater von seiner Firma eine vollständige Anleitung, mit welcher man die Basics für das Programmieren des Spiels «Space Invaders» mit Java lernen kann. Diese Anleitung erklärt einem alle verschiedenen Methoden, die man beim Programmieren benötigt. Sie erklärt einem ebenfalls, wie man vorgeht und macht es auch für Anfänger verständlich. Dadurch fühlte ich mich sicherer und motivierter, dieses Projekt in Angriff zu nehmen.

2. Planung und Vorbereitung

Die Planung meines Projekts war ein sehr wichtiger Teil davon. Sie half mir, während meines gesamten Projekts Ordnung und Struktur zu behalten. Ich habe mich bis am Ende sehr stark an ihr orientieren können, obwohl ich manchmal auch etwas weniger Zeit als geplant brauchte.

Zuerst musste ich einen groben Zeitplan erstellen, welcher den groben Umfang meines Projekts darstellen sollte. In diesem Zeitplan habe ich beschrieben, wie lange ich für das Installieren von Java brauche, wie lange das Erlernen der Programmiersprache dauert, wie lange das Einarbeiten dauert sowie das eigentliche Programmieren und das Testen des Spiels.

Bei dem Erstellen des Zeitplans musste ich darauf achten, dass ich genügend Zeit einplane für die einzelnen Schritte, um unerwartete Fehler, die beim Programmieren auftreten können, zu beheben. Dieser Zeitplan half mir dabei mich zu orientieren, damit ich nicht zu viel Zeit an manchen Schritten verliere.

Da ich bereits eine vollständige Anleitung für das Lernen von Java erhalten habe, war das Projekt für mich um einiges einfacher, da ich direkt mit dem Projekt anfangen konnte und nicht viel Zeit mit der Vorbereitung und Planung verbringen musste.

Die Audiodateien sowie die Bilder konnte ich ebenfalls von der Anleitung übernehmen, was mir sehr viel Zeit ersparte, da ich sie nicht selbst erstellen oder finden musste und so mehr Zeit hatte für das Erlernen der Programmiersprache und das Programmieren.

Ein weiterer wichtiger Teil meiner Vorbereitung waren die User Stories, in denen ich festgehalten habe, was ich in meinem Spiel haben möchte. Die User Stories halfen mir dabei meine Ziele für das Spiel im Blick zu behalten und diese Schritt für Schritt umzusetzen.

Wie oben bereits erwähnt, konnte ich mir viel Arbeit ersparen, da ich die Bilder und Audios nicht suchen oder erstellen musste und mir die Anleitung ebenfalls bereits gegeben wurde. So habe ich wenig Zeit mit der Vorbereitung des Projekts verloren und konnte früh mit dem Lernen der Programmiersprache anfangen.

Ich habe gelernt, wie wichtig eine gute Planung in einem Projekt ist, da ich ohne sie wahrscheinlich den Überblick sehr schnell verloren hätte.

3. Einzelne Arbeitsschritte

3.1 Grundlagen Java lernen

Ich habe die Grundlagen von Java mithilfe eines Moduls (https://moodle.bbcag.ch/course/view.php?id=1032), welches vom Unternehmen meines Vaters ist, gelernt.

Es wurden sehr viele wichtige Grundlagen behandelt, wie zum Beispiel die primitiven Datentypen. Diese sind essenziell für das Programmieren mit Java. Im Bild unten sieht man die verschiedenen primitiven Datentypen.

PRIMITIVE DATENTYPEN

Java hat 8 primitive Datentypen, die für einfache Werte wie Ganzzahlen, Dezimalzahlen und Zeichen verwendet werden:

Datentypname	Kurzbeschreibung	Grösse	Wertebereich	Beispiel
byte	Ganzzahl mit Vorzeichen	8 Bit	-128 bis 127	`byte b = 100;`
short	Ganzzahl mit Vorzeichen	16 Bit	-32,768 bis 32,767	`short s = 10000;`
int	Ganzzahl mit Vorzeichen	32 Bit	-2,147,483,648 bis 2,147,483,647	`int i = 100000;`
long	Ganzzahl mit Vorzeichen	64 Bit	-9,223,372,036,854,775,808 bis 9,223,372,036,854,775,807	`long l = 100000L;`
float	Fliesskommazahl	32 Bit	-	`float f = 10.5f;`
double	Fliesskommazahl	64 Bit	-	`double d = 10.5;`
char	Genau ein Unicode Zeichen	16 Bit	0 bis 65,535 (Unicode-Zeichen)	`char c = 'A';`
boolean	Wahrheitswert	1 Bit (theoretisch, aber tatsächliche Grösse kann variieren)	true oder false	`boolean bool = true;`

Die Vererbung konnte man ebenfalls in dem Modul lernen. Diese war essenziell für mein Projekt, da man mit ihr den Code übersichtlich und ordentlich halten kann. Ich habe die Vererbung oft in meinem Code gebraucht, da der Code sonst schnell unübersichtlich werden würde.

3.2 Einarbeiten in JavaFX

Zuerst musste ich ein JavaFX-Projekt mit Gradle aufsetzen, in welchem ich Code ausprobieren konnte.

Zum Beispiel habe ich hier einen Code geschrieben, der die Applikation startet und ein Fenster anzeigt.

```
import javafx.application.Application;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage stage) {
        stage.setTitle("Space Invaders");
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Für solche Testcodes, habe ich das Wissen von dem Modul benutzt, oder Hilfe von meiner Begleitperson über Teams erhalten.

Ein weiterer sehr wichtiger Code war dieser:

```
public void switchScene(Stage stage, Scene newScene) {
    stage.setScene(newScene);
}
```

Mit dem Code oben kann man von einer Szene zu einer anderen wechseln, also von einem Fenst zum nächsten. Dieser code war sehr wichtig, da ich mehrere Szenen in meinem Spiel habe.

Ich habe ebenfalls einen einfachen Code für einen Game Loop geschrieben. Ein Spiel funktioniert fast immer mit einem Game Loop. Ein Game Loop tut ständig folgendes machen:

- Spielzustand aktualisieren
- Objekte an der richtigen Stelle zeichnen

In JavaFX kann man für einen Game Loop den AnimationTimer verwenden.

Rechts ist der kurze Code für einen Game Loop:

```
AnimationTimer gameLoop = new AnimationTimer() {

    @Override
    public void handle(long now) {

        update();
        draw();

    }
};

gameLoop.start();
```

3.3 Applikation implementieren & testen

Zuerst musste ich ein neues JavaFX-Projekt erstellen.

1. Projekt starten und Navigator erstellen

In diesem Schritt habe ich einen Navigator umgesetzt, mit welchem man zwischen den verschiedenen Szenen wechseln kann, also zwischen dem Startbildschirm, dem Spiel und dem GameOver-Bildschirm.

2. Spaceship & Bewegung

Ich habe in diesem Schritt die Klasse «Spaceship» erstellt, das Bild eingefügt und die Bewegung mit den Pfeiltasten umgesetzt.

Zuerst war die Bewegung FPS-abhängig, was dazu führte, dass sich das Raumschiff je nach Computer unterschiedlich schnell bewegte. Dies war der **falsche** Code:

« x += SPEED; »

Um dieses Problem zu beheben habe ich die «deltaTime» verwendet. Dadurch wurde die Bewegung zeitbasiert und unabhängig von der Leistung des Computers. Dies war nun der **richtige** Code:

« x += SPEED * deltaTime; »

Zusätzlich habe ich eine Randbegrenzung eingebaut, damit sich das Raumschiff nur innerhalb des Fensters bewegen kann.

3. Aliens & Bomben

In diesem Schritt habe ich die Aliens und Bomben hinzugefügt. Ich habe die Aliens in einer Grid-Anordnung positioniert und sie zufällig Bomben abwerfen lassen. Zuerst standen alle Aliens auf der gleichen Position, wie der folgende **falsche** Code zeigt:

« alien.setX(100);
alien.setY(100); »

Um dieses Problem zu beheben, habe ich Schleifen für Reihen und Spalten verwendet und feste Abstände für X- und Y-Position definiert. Der **richtige** Code lautet:

« alien.setX(50 + col * 60);
alien.setY(50 + row * 50); »

4. Laser & Cooldown-System

In diesem Schritt habe ich die Laser hinzugefügt. Anfangs konnte man diese ohne Einschränkungen schiessen, was dazu führte, dass unendlich viele Laser gleichzeitig auf dem Bildschirm waren. Hier ist der **falsche** Code:

« lasers.add(new Laser(x, y)); »

Um das zu verhindern, habe ich ein Cooldown-System hinzugefügt, mit dem der Spieler erst wieder schiessen kann, wenn eine bestimmte Zeit seit dem letzten Schuss vergangen ist. Hier ist eine verkürzte Version des **richtigen** Codes:

« if (timeSinceLastShot > LASER_SHOT_TIME) shootLaser(); »

5. Kollisionen & Spielende

In diesem Schritt habe ich die Kollision zwischen Laser, Aliens und Bomben umgesetzt und ein Spielende hinzugefügt.

Anfangs habe ich versucht, die Kollision direkt über die Position zu prüfen. Das führte dazu, dass die Kollisionen fast nie erkannt wurden. Hier der **falsche** Code:
« if (laser.getX() == alien.getX() && laser.getY() == alien.getY()) { ... } »

Um das zu beheben, habe ich die Methode «collidesWith()» verwendet, die überprüft, ob sich die Bounding Boxes der Objekte überschneiden. Hier der **richtige** Code:

```
« for (Laser laser : lasers) {
for (Alienship alien : aliens) {
if (laser.collidesWith(alien)) {
alien.destroy();
laser.destroy();
}
}
} »
```

6. Restart & Sound

In diesem letzten Schritt habe ich die Möglichkeit zum Neustarten des Spiels sowie Hintergrundmusik und Soundeffekte eingebaut.

Zuerst wollte ich den Sound so laden, hier der **falsche** Code:
« Media media = new Media("sound.wav"); »

Dies funktionierte nicht, da der Pfad zum Audiofile nicht korrekt aufgelöst wurde. Um das zu beheben, habe ich die Datei in eine URI (URI= Uniform Resource Identifier) umgewandelt und einen MediaPlayer verwendet. Hier der **richtige** Code:

```
« Media media = new Media(file.toURI().toString());
MediaPlayer player = new MediaPlayer(media);
player.play(); »
```

4. Rückblick / Fazit

Am Anfang meines Projektes hatte ich mühe mit dem lernen der Programmiersprache, da viele Fachbegriffe in den Erklärungen verwendet wurden und ich noch fast keine Erfahrung im Programmieren hatte. Je länger ich jedoch die Anleitung las, desto klarer wurde sie, da sich die Fachbegriffe wiederholten und ich dadurch immer besser verstand, wofür jeder Teil tatsächlich gedacht ist.

Im Verlaufe meines Projekts habe ich ausserdem immer wieder gelernt, wie wichtig Ordnung und Struktur sind, sowohl beim Programmieren als auch beim Dokumentieren. Ohne klare Struktur wäre das Projekt deutlich schwerer gewesen. Zum Beispiel habe ich im ersten Schritt der Implementierung alles direkt in die Main Klasse geschrieben. Dadurch wurde der Code sehr schnell unübersichtlich, sodass ich ihn neu organisieren musste. Diese Erfahrung hat mir gezeigt, wie wichtig Ordnung und Struktur ist.

Ich finde es erstaunlich, dass ich am Anfang meines Projektes keine Ahnung vom Programmieren hatte und ich nun am Ende des Projektes, ein einfaches Spiel selber programmieren kann.

Während des Projekts stiess ich immer wieder auf Situationen, in denen ich überfordert war. In diesen Situationen half mir die regelmässige Kommunikation mit meiner Begleitperson über Teams sehr. Durch diese Hilfe, konnte ich nicht nur Verständnis aufbauen, sondern viele Probleme später auch selbständig lösen. Gleichzeitig motivierte mich dies weiter zu machen, da ich wusste, dass ich jederzeit Hilfe erhalten konnte.

Abschliessend muss ich sagen, das ich zu beginn meines Projekts, das Lernen von der Programmiersprache deutlich unterschätzt hatte und ich nicht erwartet hätte, das ich so viel Zeit dafür investieren müsste.

Dennoch bin ich sehr zufrieden mit dem Endergebnis, da ich sehr viele meiner Ideen umsetzen konnte. Ebenfalls konnte ich durch mein Projekt viel Erfahrung für meine zukünftige Lehre bereits sammeln, da ich dort häufig mit dem Programmieren konfrontiert werden werde.

5. Quellenverzeichnis

Kontaktpersonen:

- Michael Hegner, Softwareentwickler, Zürich

Internetadressen:

- Moodle Modul für das Lernen der Programmiersprache:
https://moodle.bbcag.ch/course/view.php?id=1032
(Kein Zugriff ohne Login-Daten).

Zugriff erhalten und erstes Mal benutzt am 09.1.2026

- Erste verwendete Internet Seite, da ich noch keinen Zugriff auf das Modul hatte:
https://www.w3schools.com/java/default.asp

Zugriff am 19.12.2025

- ChatGPT:

https://chatgpt.com/

Zugriff am 22.3.2026

- GitHub, hier ist der gesamte Code gespeichert:

https://github.com/EpicAgent/gians-space-invaders.git

Zugriff am 22.3.2026